

Manual de Programador – Ternurines

1. Objetivo del documento

Este manual técnico está dirigido a desarrolladores que darán mantenimiento o ampliarán el sistema Ternurines. Describa la arquitectura, la estructura del código, los componentes principales, el flujo de datos y las pautas de implementación.

2. Alcance

Incluye información relevante para:

- Configuración del entorno de desarrollo
- Comprensión de la arquitectura backend y frontend
- Descripción de la base de datos y entidades principales
- Pasos de despliegue local
- Reglas de desarrollo y buenas prácticas.

3. Descripción general de la solución

Ternurines es una aplicación web de gestión veterinaria compuesta por:

- Un backend Java basado en Spring Boot
- Persistencia en PostgreSQL
- Frontend estático entregado desde el servidor
- Contenedores Docker para implementación local

3.1 Objetivos de la aplicación

- Gestionar la programación de citas veterinarias
- Registrar clientes y sus mascotas
- Mantener un inventario de productos y medicamentos.
- Registrar el historial médico de las mascotas.
- Soportar roles diferenciados: administrador, recepcionista, veterinario y cliente.

4. Estructura del proyecto

4.1 Archivos y carpetas principales

- `pom.xml` : archivo de Maven que define dependencias, complementos y propiedades del build.
- `docker-compose.yml` : define los servicios de la aplicación y la base de datos PostgreSQL.
- `src/main/java/com/ternurines` : paquete raíz del código Java.
- `src/main/resources/application.properties` : configuración de Spring Boot y datos de conexión.
- `src/main/resources/db/init.sql` : script de creación de esquema y datos de inicialización.
- `src/main/resources/static` : recursos frontend estáticos (HTML, CSS, JS).

4.2 Paquetes principales

- `features.auth` : autenticación y autorización básica.
- `features.citas` : gestión de citas médicas.
- `features.clientes` : manejo de datos de clientes.
- `features.dashboard` : puntos finales para paneles y datos agregados.
- `features.historial` : gestión de historial médico.
- `features.inventario` : inventario de productos y medicamentos.
- `features.mascotas` : registro y consulta de mascotas.
- `features.operations` : operaciones adicionales o transaccionales.
- `features.servicio` : catálogo y gestión de servicios veterinarios.

5. Arquitectura del backend

5.1 Modelo de aplicación

La aplicación usa el modelo clásico de Spring Boot:

- Controladores (`@RestController`) para exponer la API REST
- Servicios (`@Service`) para la lógica de negocio
- Repositorios o DAOs (`@Repository`) para acceder a datos

5.2 Clase principal

- `com.ternurines.TernurinesApplication` : clase que arranca el contexto de Spring Boot mediante `SpringApplication.run()` .

5.3 Patrones de diseño

- Separación de responsabilidades entre capa de presentación, capa de negocio y acceso a datos.
- Uso de servicios para orquestar operaciones de negocio.
- Uso de controladores REST para exponer rutas HTTP.

6. API y puntos finales principales

6.1 Autenticación

- `POST /api/auth/login`
 - Solicitud: JSON con `correo` y `contrasena`
 - Respuesta: objeto con datos de usuario, rol y token de sesión.

6.2 Gestión de clientes

- `GET /api/clientes`
- `POST /api/clientes`
- `PUT /api/clientes/{id}`
- `DELETE /api/clientes/{id}`

6.3 Gestión de mascotas

- `GET /api/mascotas`
- `POST /api/mascotas`
- `PUT /api/mascotas/{id}`
- `DELETE /api/mascotas/{id}`

6.4 Gestión de citas

- `GET /api/citas`
- `POST /api/citas`
- `PUT /api/citas/{id}`
- `DELETE /api/citas/{id}`

6.5 Gestión de inventarios y servicios

- `GET /api/inventario`

- GET /api/servicios
- POST /api/servicios
- PUT /api/servicios/{id}

Nota: La implementación puede variar en detalles de cada controlador y rutas adicionales. Revise las clases `*Controller.java` para confirmar la firma exacta de cada punto final.

7. Base de datos

7.1 Esquema principal

El script `src/main/resources/db/init.sql` crea las tablas principales y relaciones:

- cliente
- veterinario
- recepcionista
- administrador
- mascota
- mascota_adopcion
- cita
- servicio
- cita_servicio
- medicamento
- historial_medico
- tratamiento
- producto
- adoptante
- adopcion

7.2 Relaciones clave

- `mascota.id_cliente` → `cliente.id_cliente`
- `cita.id_mascota` → `mascota.id_mascota`
- `cita.id_veterinario` → `veterinario.id_veterinario`
- `cita.id_recepcionista` → `recepcionista.id_recepcionista`
- `cita_servicio` une citas y servicios
- `historial_medico.id_mascota` → `mascota.id_mascota`
- `tratamiento.id_historial` → `historial_medico.id_historial`

- `producto.id_administrador` y `medicamento.id_administrador` referencian administrador

7.3 Datos de prueba

El script incluye datos iniciales para:

- usuarios de cada rol
- clientes y mascotas
- servicios veterinarios
- productos y medicamentos
- citas de ejemplo
- historial médico y tratamientos

8. Configuración del entorno

8.1 Propiedades de Spring Boot

Archivo: `src/main/resources/application.properties`

- `spring.datasource.url=jdbc:postgresql://${DB_HOST:db}:${DB_PORT:5432}/${DB_NAME:ternurines}`
- `spring.datasource.username=${DB_USER:ternurines}`
- `spring.datasource.password=${DB_PASSWORD:ternpass}`
- `spring.datasource.driver-class-name=org.postgresql.Driver`
- `spring.jpa.hibernate.ddl-auto=none`
- `spring.main.allow-bean-definition-overriding=true`
- `server.port=8080`

8.2 Variables de entorno en Docker Compose

El despliegue local usa Docker Compose para gestionar:

- servicio de la aplicación Java
- servicio de base de datos PostgreSQL

9. Frontend

9.1 Estructura de recursos estáticos

- `src/main/resources/static/index.html` : pantalla de login.
- `src/main/resources/static/admin-dashboard.html` : panel del administrador.
- `src/main/resources/static/receptionist-dashboard.html` : panel de recepcionista.
- `src/main/resources/static/veterinary-dashboard.html` : panel de veterinario.
- `src/main/resources/static/user-dashboard.html` : panel de cliente.
- `src/main/resources/static/assets/css` : estilos globales.
- `src/main/resources/static/assets/js` : lógica de interacción.

9.2 Flujo de autenticación

- `assets/js/auth.js` maneja el envío del login y la redirección.
- El formulario envía un `POST` a `/api/auth/login` con el payload JSON:
 - `correo`
 - `contrasena`
- Al recibir la respuesta, el frontend normaliza el rol y redirige según el mapa de dashboards.

9.3 Manejo de sesión en el cliente

- `SessionManager` persiste el token y los datos del usuario en `localStorage`.
- `handleLogout()` elimina la sesión y regresa al login.
- El frontend verifica si existe sesión activa en la carga de páginas y redirige al dashboard si corresponde.

10. Proceso de ejecución

10.1 Ejecución con Docker Compose

Desde la raíz del proyecto:

```
docker compose up --build
```

Para reiniciar y limpiar datos de volumen:

```
docker compose down -v
```

10.2 Ejecución con Maven

Si prefiere ejecutar sin Docker, configure PostgreSQL local y use:

```
mvn spring-boot:run
```

10.3 Acceso a la aplicación

- URL de la aplicación: `http://localhost:8080`
- El frontend estático se sirve directamente desde Spring Boot.

11. Flujo de desarrollo

11.1 Añadir un nuevo módulo

1. Crear paquete bajo `src/main/java/com/ternurines/features`.
2. Definir modelo de datos, servicio y controlador.
3. Añadir repositorio o acceso a datos con anotación `@Repository`.
4. Escribir la lógica de negocio en la capa de servicio.
5. Exponer endpoints en `@RestController`.
6. Actualizar el frontend estático si requiere nuevas vistas.

11.2 Añadir una nueva entidad en la base de datos

1. Extender `init.sql` con la nueva tabla y restricciones.
2. Actualizar los repositorios y servicios.
3. Registrar casos de prueba si aplica.
4. Verificar con un despliegue limpio de Docker Compose.

12. Recomendaciones y buenas prácticas

- Mantenga separada la lógica de negocio de la presentación.
- Evite consultas SQL directamente en los controladores.
- Utilice tipos de datos compatibles con PostgreSQL.
- Use nombres claros y consistentes para rutas y métodos.
- Comente el código donde la lógica no sea obvia.
- Asegure la consistencia entre el frontend y la API.

13. Consideraciones de seguridad

- En producción, reemplace las contraseñas por valores cifrados.
- Aplique HTTPS para el tráfico web.
- Desactive `spring-boot-devtools` en entornos productivos.
- Asegure los endpoints con autenticación y autorización adecuada.

14. Referencias técnicas

- Clase principal: `src/main/java/com/ternurines/TernurinesApplication.java`
- Configuración: `src/main/resources/application.properties`
- Base de datos: `src/main/resources/db/init.sql`
- Frontend: `src/main/resources/static` y `assets/js`
- Dependencias Maven: `pom.xml`

15. Entrega y documentación

Este manual sirve como documento de transferencia técnica para el equipo de desarrollo. Contiene la información necesaria para entender la solución, desplegarla localmente y continuar con su mantenimiento.